

Case Study: ModaCo — Promotion Management API

Submission: Please provide a repository link containing your code, your database schema (DDL), a brief document explaining your architectural decisions (ADR), and the **AI Usage Appendix (Form 5)**.

AI Usage Policy:

We allow and actively encourage the use of AI tools (Claude, ChatGPT, Copilot, Cursor, etc.). However, submitting Form 5 (AI Appendix) alongside your solution is mandatory. During the technical interview, we will thoroughly discuss how you collaborated with AI, and specifically how you identified and corrected any architectural or logical mistakes it made.

1. Domain and Business Rules

ModaCo requires an internal API to manage its product catalog and promotions (e.g., seasonal discounts, flash sales).

- **Product:** Has a name, category, SKU, base price, and stock quantity.
- **Promotion:** Has a name, discount type (percentage or fixed amount), value, and valid dates (start/end). It can be assigned to a specific product or an entire category.
- **Business Rule:** A product can have at most **one** active promotion at a time. When listing products, the system must return the calculated **effective price** (the price after applying the active promotion), not just the base price. Promotion conflicts must be handled logically.

2. Core Expectations

Using Node.js, Express, and TypeScript (database and ORM choices are up to you), develop REST API endpoints that provide the following capabilities:

- List products (Must include filtering by category, pagination, and **sorting by effective price**).
- Get single product details (This is the most highly trafficked endpoint in the system).
- Create, cancel, and assign promotions to products or categories.

3. Critical Business Scenarios (Scalability Test)

When designing your system, you are expected to consider ModaCo's operational realities and build your architecture accordingly. An API that merely performs standard CRUD operations will not be accepted; the structural solutions you provide for the two scenarios below will be the core of your evaluation:

Scenario A: "Massive Data Ingestion" (Vendor Sync & Serverless Constraint)

ModaCo receives weekly large files containing 500,000+ rows of new product/pricing data from vendors. You must integrate a mechanism into your system to ingest this data. However, this external data cannot be written directly to the database; every record must pass through internal dynamic pricing rules (an application-layer process) before being saved.

Constraint: To optimize costs, this data ingestion/processing unit of your system must be designed to run on a **Serverless Consumption Plan** (e.g., Azure Functions, AWS Lambda). Due to the nature of this environment:

- Your processes have a strict "Timeout" limit (e.g., a few minutes).
- Server memory is highly restricted.
- The processor halts as soon as the HTTP request completes (Stateless).

Under these constraints, design and implement an approach to successfully ingest a 500,000-row file into the system without crashing your application or having the process cut off midway.

Scenario B: "Flash Sales" (System Load Distribution)

ModaCo frequently runs instant flash sales, such as "50% Off All Accessories". The second a campaign is created, it instantly affects over 50,000 products. During flash sales, the GET /products listing endpoint (the storefront) experiences massive read traffic. Additionally, if a brand new product is added to the "Accessories" category while the flash sale is active, that product must automatically benefit from the discount. Develop a strategy to prevent the system from encountering bottlenecks during these simultaneous heavy read and write operations.

4. Architectural Decisions and AI Documentation (Mandatory)

Please include the following two text (or Markdown) files in the root directory of your project submission:

1. Architecture Decision Record (ADR.md):

Explain the architectural approach you took, especially when solving Scenario A and Scenario B. Technically defend the design choices (patterns, tools, database structures) you made and discuss the technical or business "trade-offs" those choices entail.

2. Form 5: AI Interaction Summary (AI_APPENDIX.md):

- Which AI tools (and models) did you use throughout the process?
- What were the 2 most critical and comprehensive prompts you provided to the AI?
- **Error Correction:** What was the biggest architectural/logical mistake you noticed in the solution the AI provided (especially for Scenario A or B), and how did you alter/steer the code to prevent the system from failing?